

Half-space Worm-Like Chain (WLC) simulations.

This document describes fortran subroutines contained in source files

```
../src/halfspace_WLC_simulations/WLC_simulation_posZ_endpoint.f  
../src/halfspace_WLC_simulations/WLC_simulation_posZ_stepwise.f
```

... which depend on inverse Langevin function contained in the source file

```
../src/math_lib/inv_Langevin_function.f
```

Examples of practical application of the above subroutines and functions are contained in fortran programs in files

```
../src/halfspace_WLC_simulations/WLC_distance_distribution.f  
../src/halfspace_WLC_simulations/WLC_concentration_ABCD.f
```

All of these programs, subroutines, and functions were used to generate distance distributions and local concentrations for the article titled "Multivalent weak contacts shape chaperone-nascent protein interactions", authored by N. Rajasekaran, D. Toptygin, T.-W. Liao, V. J. Hilser, T. Ha, C. M. Kaiser, to be published in Proceedings of the National Academy of Sciences (2026). In that article half-space WLC simulations were intended to model the nascent polypeptide chain emerging from the ribosome exit tunnel. The ribosome surface was approximated as the $Z=0$ plane; the nascent chain was not allowed to enter the half-space $Z \leq 0$ that was occupied by the ribosome.

The subroutines in source files `WLC_simulation_posZ_endpoint.f` and `WLC_simulation_posZ_stepwise.f` are based on the coarse-grained model that represents the WLC as a chain consisting of segments $\vec{r}_i$, $i=1,2,...,NM$, where NM is the Number of Monomers (such as amino acids in the nascent chain). The length of each segment equals BL . The persistent length of the WLC equals PL . The values of PL (real*8), BL (real*8), and NM (integer*4) must be specified when calling the subroutine in source file `WLC_simulation_posZ_endpoint.f`:

```
CALL WLC_SIMULATION_POSZ(X,Y,Z,PL,BL,NM)
```

The subroutine calculates the coordinates X , Y , Z (all real*8) of the end segment and returns control to the calling program. The WLC always starts at the coordinates (0,0,0) and the segment before the first one is pointing in the direction (0,0,1). Each call to the subroutine will result in new values of X , Y , Z , since a random number generator is used to calculate the direction of each segment. Note, that the subroutine in `WLC_simulation_posZ_endpoint.f` does not return the coordinates of intermediate segments; it returns only the coordinates of the end segment. If the user needs the coordinates of all segments, then the subroutine in the source file `WLC_simulation_posZ_stepwise.f` must be used. This subroutine must be first initialized:

```
CALL WLC_SIMULATION_POSZ_SETUP(X,Y,Z,PL,BL)
```

After the initialization the coordinates X , Y , Z are all zero, as at the start of the WLC. Then the main program must make NM calls (one per segment) to a different entry point of the subroutine:

```
CALL WLC_SIMULATION_POSZ_ONESTEP(X,Y,Z)
```

After each call to the subroutine entry point, it returns the coordinates of the next segment; the X , Y , Z values can be used by the calling program, but they must not be altered between calls.

The operation of both subroutines is based on the following mathematical model.

The direction of segment i is parallel to a unit vector $\vec{d}_i$ that is defined as

$$\vec{d}_i = \frac{\vec{r}_i}{|\vec{r}_i|} = \frac{\vec{r}_i}{BL} \quad (1)$$

Worm-like chain model assumes that the bending energy is directly proportional to the sum over all segments of the dot products of the unit vectors $\vec{d}_{i-1}$ and $\vec{d}_i$:

$$U = -\varepsilon \sum_{i=1}^N (\vec{d}_{i-1} \cdot \vec{d}_i) \quad (2)$$

The dot product of two unit vectors equals cosine of the angle between them,

$$(\vec{d}_{i-1} \cdot \vec{d}_i) = \cos \theta_i \quad (3)$$

therefore the bending energy can be also expressed in terms of the angles θ_i :

$$U = -\varepsilon \sum_{i=1}^N \cos \theta_i \quad (4)$$

The vector $\vec{d}_{i-1}$ and the angle θ_i are insufficient to uniquely define the direction of the unit vector $\vec{d}_i$: one more angle φ_i is needed. One can think of θ_i and φ_i as the bond angle and the dihedral angle (although the bond angle actually equals $\pi - \theta_i$). According to Boltzmann distribution, at an absolute temperature T the probability density of the angles θ_i and φ_i is an exponential function of the bending energy,

$$p(\theta_i, \varphi_i) d\theta_i d\varphi_i = C \exp\left(\frac{\varepsilon \cos \theta_i}{k_B T}\right) \sin \theta_i d\theta_i d\varphi_i \quad (5)$$

where C is a normalization factor, k_B is the Boltzmann constant, and $\sin \theta_i d\theta_i d\varphi_i$ is an elementary solid angle. Since the expression on the right-hand side of equation (5) is independent of φ_i , both sides of this equation can be integrated from 0 to 2π by $d\varphi_i$, yielding the probability density of the angle θ_i :

$$p(\theta_i) d\theta_i = 2\pi C \exp\left(\frac{\varepsilon \cos \theta_i}{k_B T}\right) \sin \theta_i d\theta_i \quad (6)$$

The normalization factor C can be calculated by equating to 1 the integral from 0 to π of the right-hand side of Equation (6); substituting the resulting value of C in equation (6) results in

$$p(\theta_i) = \frac{\xi \exp(\xi \cos \theta_i) \sin \theta_i}{\exp(\xi) - \exp(-\xi)} \quad (7)$$

where ξ is a combination of three parameters,

$$\xi = \frac{\varepsilon}{k_B T} \quad (8)$$

Ensemble mean $\langle \cos \theta_i \rangle$ can be obtained by integrating $\cos \theta_i p(\theta_i)$ from 0 to π , which gives

$$\langle \cos \theta_i \rangle = \text{Langevin}(\xi) \quad (9)$$

where $\text{Langevin}(\xi)$ is the well known Langevin function, $\text{Langevin}(\xi) = \coth(\xi) - 1/\xi$, and $\coth(\xi)$ is a hyperbolic cotangent. According to the worm-like chain model the ensemble mean $\langle \cos \theta_i \rangle$ must also satisfy the following relation:

$$\langle \cos \theta_i \rangle = \exp(-BL/PL) \quad (10)$$

Combining equations (9) and (10) we obtain

$$\text{Langevin}(\xi) = \exp(-BL/PL) \quad (11)$$

Resolving equation (11) with respect to ξ allows one to express the new parameter ξ in terms of the known parameters BL and PL . In practice this requires calculating the inverse Langevin function, which is accomplished using Newton-Raphson method (this is a challenging task because both the Langevin function and its first derivative experience catastrophic cancellation in the vicinity of $\xi=0$). A fortran function in the source file `inv_Langevin_function.f` calculates the inverse Langevin function with the full precision of `real*8` numbers (described later in this document).

The actual algorithm for simulating a WLC consists of a few preparatory operations followed by NM steps during which the WLC is gradually built up one segment at a time. The preparatory operations consist of: (i) calculating the parameter ξ from the parameters BL and PL as described above, (ii) defining the initial direction $\vec{d}_0 = (0,0,1)$ which is normal to the $Z=0$ plane and pointing in the $+Z$ direction, and (iii) defining the radius-vector $\vec{R}_0 = (0,0,0)$ that determines the starting coordinates of the WLC.

Algorithm step number i ($i=1,2,\dots,NM$) consists of the following operations. First, two unit vectors are constructed that are both normal to $\vec{d}_{i-1}$ and also normal to each other. Toward this goal the algorithm finds the component of the vector $\vec{d}_{i-1}$ that is smallest in absolute value and constructs a unit vector $\vec{e}_{i-1}$ parallel to this component. For example, if the x component of vector $\vec{d}_{i-1}$ is the smallest in magnitude, then $\vec{e}_{i-1} = (1,0,0)$, if the y component of vector $\vec{d}_{i-1}$ is the smallest in magnitude, then $\vec{e}_{i-1} = (0,1,0)$, etc. After that the vector $\vec{e}_{i-1}$ is orthogonalized with the unit vector $\vec{d}_{i-1}$ and normalized to a unit length:

$$\vec{f}_{i-1} = \frac{\vec{e}_{i-1} - (\vec{d}_{i-1} \cdot \vec{e}_{i-1}) \vec{d}_{i-1}}{|\vec{e}_{i-1} - (\vec{d}_{i-1} \cdot \vec{e}_{i-1}) \vec{d}_{i-1}|} \quad (12)$$

Vector $\vec{f}_{i-1}$ is a unit vector and it is normal to $\vec{d}_{i-1}$. The third unit vector $\vec{g}_{i-1}$ that is normal to both $\vec{d}_{i-1}$ and $\vec{f}_{i-1}$ is calculated as the cross product

$$\vec{g}_{i-1} = [\vec{d}_{i-1} \times \vec{f}_{i-1}] \quad (13)$$

Next, a random value for $\cos \theta_i$ is generated

$$\cos(\theta_i) = \frac{1}{\xi} \ln \left[\exp(\xi) - [\exp(\xi) - \exp(-\xi)] * RND \right] \quad (14)$$

where RND is a pseudo-random number with values uniformly distributed between 0 and 1, generated by the intrinsic subroutine `RANDOM_NUMBER(RND)` in fortran. It can be shown that the $\cos\theta_i$ values generated using equation (14) strictly comply with the probability distribution in equation (7). The value of $\sin\theta_i$ is calculated as

$$\sin(\theta_i) = \sqrt{1 - \cos^2(\theta_i)} \quad (15)$$

Next, a random value for ϕ_i is generated; this angle is uniformly distributed between $-\pi$ and $+\pi$.

$$\phi_i = 2\pi \left(RND - \frac{1}{2} \right) \quad (16)$$

where RND is a new pseudo-random number with values uniformly distributed between 0 and 1, generated by the intrinsic subroutine `RANDOM_NUMBER(RND)` in fortran. Next, the new unit vector $\vec{d}_i$ is calculated, followed by the new vectors $\vec{r}_i$ and $\vec{R}_i$:

$$\vec{d}_i = \cos(\theta_i) \vec{d}_{i-1} + \sin(\theta_i) \cos(\phi_i) \vec{f}_{i-1} + \sin(\theta_i) \sin(\phi_i) \vec{g}_{i-1} \quad (17)$$

$$\vec{r}_i = BL \vec{d}_i \quad (18)$$

$$\vec{R}_i = \vec{R}_{i-1} + \vec{r}_i \quad (19)$$

In the case of unconstrained simulations this would represent the last operation of step number i . Constrained simulations include one more operation that will be discussed later.

The unconstrained worm-like chain model predicts the following correlation between the direction $\vec{d}_i$ and $\vec{d}_{i+m}$:

$$\langle \vec{d}_i \cdot \vec{d}_{i+m} \rangle = \exp(-m BL / PL) \quad (20)$$

This relation must hold for any values of i and m . To test the algorithm of unconstrained WLC simulations, 10^8 WLCs were simulated, and the mean values of the dot products of vectors $\vec{d}_i$ and $\vec{d}_{i+m}$ were calculated for a wide range of m values. It was confirmed that the relation in equation (20) holds with great accuracy (the accuracy is determined only by the number of WLCs simulated). This showed that the unconstrained algorithm works exactly as expected.

In the case of constrained WLC simulations equation (20) is not expected to hold, however, all the operations that constitute the algorithm step number i are still applicable and justified. The radius-vector $\vec{R}_i$ generated at the end of step i consists of three components: X_i , Y_i , Z_i . The component Z_i must be positive for all $i=1,2,\dots,NM$. If at the end of the step number i the value Z_i is negative or equals zero, then this WLC is not allowed. Two actions can be considered in this case. If the WLC is considered to be rigid so that it would rather move as a whole than change its conformation, then upon encountering $Z_i \leq 0$ the algorithm should be restarted at $i=1$ and a new WLC should be generated from the start. However, if the WLC is considered to be flexible so that it would rather change the angles θ_i and ϕ_i than move away from the $Z=0$ surface, then upon encountering $Z_i \leq 0$ the algorithm should be sent back just one step to $i=i-1$ and repeat the last step as many times as it takes until $Z_i > 0$ is obtained. The latter approach is used by the subroutines described here.

Calculating end-to-end distance distributions.

The program in the source file `WLC_distance_distribution.f` represents an example of using the subroutine in the source file `WLC_simulation_posZ_endpoint.f`. The program calculates the probability density of the end-to-end distance distribution for a WLC under the constraint $Z > 0$. This program should be edited by any text editor (like `gedit` or `mousepad` or `vi` or `nano`) and the desired values should be assigned to the parameters *PL*, *BL*, *NM*, *NWLC*, *DR*, *NBIN*. The original file contains the parameter values for a 259 segment long worm-like chain with 0.36 nm contour length per segment and 0.5 nm persistence length. The distribution is binned in 10000 bins of 0.04 nm bin width and a total of 100000000 WLCs are simulated to reduce random noise in the distance distribution:

```
C          Edit the values of the following six parameters
C          before using the command "make distance_distribution"
C
      PARAMETER(PL=0.50D0)      ! Persistence length (nm)
      PARAMETER(BL=0.36D0)      ! Contour length per monomer (nm)
      PARAMETER(NM=259)         ! Number of monomers in the polymer
      PARAMETER(NWLC=100000000) ! Number of WLC simulated; greater
C                               !     NWLS produces smoother distribution
      PARAMETER(DR=0.04D0)      ! Bin width for the distance distribution
      PARAMETER(NBIN=10000)     ! Total number of bins
```

After editing the values, change to the directory containing Makefile and use the command

```
make distance_distribution
```

This will create a file `WLC_distance_distribution` in the current directory. Run it:

```
./WLC_distance_distribution
```

This may take some time, which is proportional to the product $NM \times NWLC$; the program prints the current percent of completion on the screen and updates it after each 0.1%. After the program finishes, you will discover an ASCII file `distribution.txt` with the distance distribution (two columns; first column = distance, second column = probability density). You can now rename the file or copy it to another directory and then repeat the above steps to calculate another distance distribution (for a different combination of parameters *PL*, *BL*, *NM*).

Calculating WLC end concentrations at multiple points in space.

The program in the source file `WLC_concentration_ABCD.f` represents an example of using the subroutine in the source file `WLC_simulation_posZ_stepwise.f`. The program calculates the dependence on the WLC length of the local millimolar concentrations of the ends of the WLCs at four points *PA*, *PB*, *PC*, *PD* with known coordinates; the WLCs are constrained by the condition $Z > 0$. This program should be edited by any text editor (like `gedit` or `mousepad` or `vi` or `nano`) and the desired values should be assigned to the parameters *PL*, *BL*, *NM*, *NWLC*, *TOROIDR*, *PAX*, *PAY*, *PAZ*, *PBX*, *PBY*, *PBZ*, *PCX*, *PCY*, *PCZ*, *PDX*, *PDY*, *PDZ*, . The original file contains the parameter values for a 260 segment long worm-like chain with 0.36 nm contour length per segment and 0.5 nm persistence length. A total of 100000000 WLCs are simulated to reduce random noise in the local concentrations, the cutoff radius for estimating the local concentration

is set to 0.2 nm from each point (*PA*, *PB*, *PC*, *PD*), the *X*, *Y*, *Z* coordinates of each point in nm are also provided as parameters:

```
C      Edit the values of the following 5+12 parameters
C      before using the command "make concentrations"
C
      PARAMETER(PL=0.50D0)      ! Persistence length (nm)
      PARAMETER(BL=0.36D0)      ! Contour length per monomer (nm)
      PARAMETER(NM=260)         ! Number of monomers in the polymer
      PARAMETER(NWLC=100000000) ! Number of WLC simulated; greater
C                                ! NWLS produces smoother distribution
      PARAMETER(TOROIDR=2.0D-1) ! Cutoff distance from the points PA, PB,
C                                ! PC, PD; equivalent of bin width in nm.
C
C      Coordinates X, Y, Z for the points PA, PB, PC, PD:
C
      PARAMETER(PAX= 2.08612D0, PAY=-1.14450D0, PAZ= 3.59757D0)
      PARAMETER(PBX= 4.00242D0, PBX=-0.62052D0, PBZ= 1.99452D0)
      PARAMETER(PCX= 5.74549D0, PCY=-2.73123D0, PCZ= 1.40088D0)
      PARAMETER(PDX= 3.75826D0, PDY=-6.71115D0, PDZ= 2.01347D0)
C
C      End of the parameters that can be edited.
```

After editing the values, change to the directory containing Makefile and use the command
 make concentrations

This will create a file WLC_concentration_ABCD in the current directory. Run it:

```
./WLC_concentration_ABCD
```

This may take some time, which is proportional to the product $NM \times NWLC$; the program prints the current percent of completion on the screen and updates it after each 0.1%. After the program finishes, you will discover four ASCII files named concentration_PA.txt, concentration_PB.txt, concentration_PC.txt, concentration_PD.txt. Each file contains two columns: first column = the number of segments (integer), second column = local millimolar concentration of the end of the WLC at this number of segments.

The program makes use of the symmetry of the problem due to which the local concentration of WLC ends depends on only two variables: $H = \sqrt{X^2 + Y^2}$ and Z . The values of H are calculated for each of the four points *PA*, *PB*, *PC*, *PD* and named *PAH*, *PBH*, *PCH*, *PDH*, respectively. NM bins are created for each of the points. At each simulation step i the vector $\vec{R}_i$ is used to calculate H_i and Z_i . If the condition $(H_i - PAH)^2 + (Z_i - PAZ)^2 \leq (TOROIDR)^2$ is satisfied, then a count is added to bin number i for point *PA*. Likewise, if the condition $(H_i - PBH)^2 + (Z_i - PBZ)^2 \leq (TOROIDR)^2$ is satisfied, then a count is added to bin number i for point *PB*, etc.. The local concentrations are calculated as the number of counts in a bin divided by the product (toroid volume $\times NWLC$); the toroid volume for point *PA* is calculated as $2\pi^2(PAH)(TOROIDR)^2$, for point *PB* as $2\pi^2(PBH)(TOROIDR)^2$, etc.. To convert concentrations from molecules per nm³ to moles per liter, it is multiplied by 10³, then by 10²⁴ nm³/liter, and divided by the Avogadro number.

Inverse Langevin function, full real*8 precision.

The fortran function in the source file `inv_Langevin_function.f` calculates the inverse of the Langevin function with full real*8 precision. By definition of inverse function, if

$$y = \text{inv_Langevin}(x) \quad (21)$$

then

$$x = \text{Langevin}(y) \quad (22)$$

where $\text{Langevin}(y)$ is the Langevin function defined as

$$\text{Langevin}(y) = \coth(y) - 1/y \quad (23)$$

and $\coth(y)$ is the hyperbolic cotangent.

The Newton-Raphson method is commonly used to numerically calculate the inverse of a function. The Newton-Raphson method requires calculating the direct function and its derivative with high precision. In the case of the Langevin function this presents a challenge because Langevin function suffers from catastrophic cancellation near $y=0$. Both $\coth(y)$ and $1/y$ tend to infinity as y approaches 0, however, the difference $\coth(y)-1/y$ tends to 0 as y approaches 0. Thus, subtracting $1/y$ from $\coth(y)$ results in poor accuracy near $y=0$. An alternative method of calculating the Langevin function is based on the continued fraction expansion

$$\text{Langevin}(y) = \frac{y}{3 + \frac{y^2}{5 + \frac{y^2}{7 + \frac{y^2}{9 + \frac{y^2}{11 + \ddots}}}}} \quad (24)$$

that follows directly from Lambert's continued fraction expression for $\tanh(y)$,¹

$$\tanh(y) = \frac{y}{1 + \frac{y^2}{3 + \frac{y^2}{5 + \frac{y^2}{7 + \frac{y^2}{9 + \frac{y^2}{11 + \ddots}}}}} \quad (25)$$

The continued fraction expansion (24) makes it possible to calculate $\text{Langevin}(y)$ with full real*8 precision for small absolute values of y , however, the greater the absolute value of y , the more fractions must be included, which increases the processor time required. This calls for a compromise between precision and processor time. For $y < -3$ and for $y > +3$ the use of the expression in equation (23) produces full real*8 precision. For $-3 \leq y \leq +3$ the expression in

¹ Wall, H. S. (1973). *Analytic Theory of Continued Fractions*. Chelsea Publishing Company. p. 349. ISBN 0-8284-0207-8.

equation (24) with just twelve fractions produces full real*8 precision [equation (24) shows only the first five of the twelve functions]. The function INV_LANGEVIN(X) switches between the expressions in equations (23) and (24) with twelve fractions at $y=-3$ and $y=+3$. Since both expressions in equations (23) and (24) represent elementary functions, their derivatives can be calculated using the common rules for finding the derivatives of elementary functions; the derivatives of the Langevin function calculated in this manner are employed by the function INV_LANGEVIN(X).

Before the Newton-Raphson method can be applied, a starting value must be assigned to y . In the case of the inverse Langevin function, the Newton-Raphson method works just fine with the starting value of $y=0$, however, the number of iterations required to find the final value can be very large in this case. The function INV_LANGEVIN(X) uses the starting value of

$$y_0 = x \frac{3-x^2}{1-x^2} \quad (26)$$

that was suggested by A. Cohen² as an approximation for the inverse Langevin function. With this starting value the number of iterations required to achieve real*8 precision does not exceed five (the most common number of iterations equals four).

Each iteration n consists of calculating Langevin(y) and its derivative dLangevin(y)/ dy as described above, and then performing two arithmetic operation

$$\Delta y_n = \frac{x - \text{Langevin}(y_{n-1})}{d\text{Langevin}(y_{n-1})/dy_{n-1}} \quad (27)$$

$$y_n = y_{n-1} + \Delta y_n \quad (28)$$

Iterations end when the condition $|\Delta y_n| \leq 10^{-13}$ is satisfied; in this case Langevin(y_n) differs from the argument x by no more than $\pm$ one unit of the smallest significant digit of the real*8 number x .

2 Cohen, A. (1991). "A Padé approximant to the inverse Langevin function". *Rheologica Acta*. **30** (3): 270–273. doi:10.1007/BF00366640.

MIT License

Copyright (c) 2026 Christian M. Kaiser and Dmitri Toptygin

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.